Part I: SummitWorks Development Pipeline Analysis

Course: WEB430

Student: Jonathan Cantu

Professor: Richard Krasso

Date: September 10, 2026

SummitWorks has many of the core elements of a modern DevOps workflow, including version control, CI/CD, automated testing, and AI-assisted tools. However, the current implementation lacks consistency, predictability, and trust. The problems described in the scenario are interconnected: long-lived branches produce larger pull requests, inconsistent reviews allow defects to move forward, unreliable tests reduce confidence in CI, slow builds weaken feedback loops, manual deployment steps increase risk, and inconsistent use of AI creates uneven results. Rather than adding more tools immediately, SummitWorks should first stabilize the workflow and establish clear practices around how changes move from development through deployment.

1. Evaluate Version Control Practices

The version control process shows problems with both Flow and Feedback. Branches that remain open too long are more likely to drift from the main branch, increasing merge conflicts and rework. Krasso (2026) emphasizes that short-lived branches reduce merge complexity, support frequent integration, and improve overall Flow.

I also believe Flow begins before a branch is ever created. In my experience, oversized branches and pull requests often trace back to user stories or bug tickets that were too broadly scoped in JIRA. Business Analysts and Product Owners should break larger requirements into smaller, clearly defined tickets and group related work under an Epic when necessary. Smaller units of work naturally lead to smaller branches, focused pull requests, faster reviews, and quicker feedback.

Pull requests should also follow consistent expectations. Each PR should explain what changed, why it changed, reference the associated ticket, and identify how the change was validated. Automated checks should handle formatting, linting, and other repetitive tasks so reviewers can focus on correctness, maintainability, design, and risk.

2. Evaluate CI Practices

SummitWorks' CI process appears unreliable because automated tests fail intermittently and builds take too long. Both conditions weaken CI because developers eventually stop trusting feedback that is slow or inconsistent. Krasso (2026) explains that CI depends on fast, reliable feedback and that slow or unpredictable pipelines encourage behaviors such as batching changes or ignoring failures.

I would establish GitHub Actions quality gates on every pull request targeting the main branch. A practical Node.js workflow could use `actions/checkout` to obtain the code and `actions/setup-node` to establish a consistent runtime, followed by `npm ci`, automated tests, and `npm run build`. The resulting build-and-test job should be configured as a required status check so a PR cannot merge when the build or tests fail. GitHub supports required checks specifically for this purpose (GitHub, 2026).

SummitWorks should also identify flaky tests, measure build duration, and investigate which stages consume the most time. Caching dependencies and parallelizing independent tests could improve feedback time without weakening the gate itself.

3. Evaluate CD Practices

The manual approval required for staging deployment introduces another break in Flow. Manual approval is not automatically inappropriate, particularly for production, but requiring it for routine staging deployments can create unnecessary waiting.

After the required CI quality gates succeed, a second GitHub Actions job could automatically deploy the validated artifact to a GitHub `staging` environment. GitHub Environments are designed to represent deployment targets such as development, staging, and production and can apply different protection rules and secrets to each environment (GitHub, 2026). This allows staging to remain highly automated while stronger approval requirements can still be preserved for production.

The goal is to make deployment repeatable. The same artifact that passed testing should move forward to staging rather than being rebuilt or deployed through a separate manual procedure.

4. Evaluate Automated Testing

Unreliable automated tests may be the most damaging problem in the current pipeline because every later stage depends on trusting their results. Automated testing should give developers confidence that a change has not broken existing behavior. When tests fail intermittently, that confidence disappears.

Krasso (2026) describes automated tests as the safety net that allows teams to move quickly without sacrificing quality. SummitWorks should track flaky tests and treat them as defects rather than accepted background noise. Regression tests should be added when defects are discovered, and test execution should be organized so developers receive meaningful feedback as early as possible.

The objective is not simply to maximize the number of tests. It is to maintain a test suite that is fast, reliable, and meaningful enough to function as a trustworthy quality gate.

5. Identify Opportunities for AI-Assisted Support

Once the underlying workflow becomes stable, intelligent agents could provide meaningful support in several areas. A code review agent could summarize pull requests, flag missing tests, identify risky changes, or point reviewers toward files requiring additional attention. This can reduce cognitive load without replacing human review.

A dependency agent could monitor libraries, create pull requests for updates, execute the existing test suite, and notify the team when an update introduces problems. However, Krasso (2026) warns that dependency automation can create more uncertainty if the team does not already trust its test coverage.

Pipeline optimization agents could analyze slow build stages, identify bottlenecks, recognize flaky tests, and suggest caching or parallelization. Monitoring agents could analyze logs and operational signals to identify unusual patterns and help developers investigate failures.

These agents should be used cautiously when decisions involve architecture, business requirements, security risk, or production impact. An agent can identify patterns and make recommendations, but human engineers should remain responsible for consequential decisions.

6. Recommended Improvements

I would improve SummitWorks incrementally. First, improve JIRA story and bug-ticket sizing so developers begin with smaller units of work. Keep branches short-lived and establish consistent PR expectations. Next, stabilize automated tests and make a GitHub Actions build-and-test workflow a required merge gate. Then reduce build time and automate staging deployment after those checks succeed.

The team should also establish clear ownership for pipeline failures so failed builds are investigated rather than ignored. Once those foundations are trustworthy, code review and pipeline optimization agents can be introduced gradually and evaluated based on whether they reduce friction.

7. Connect to the Three Ways

These recommendations directly support Flow, Feedback, and Continuous Learning.

Flow starts with smaller JIRA stories and bug tickets and continues through short-lived branches, focused PRs, faster builds, and automated staging deployments. Breaking work into smaller chunks at the source reduces the chance that oversized work becomes an oversized branch or PR later.

Feedback improves when every PR automatically runs required build and test checks before merge. A developer learns quickly whether a change broke the application rather than discovering the problem later in staging or production.

Continuous Learning improves when SummitWorks examines patterns in flaky tests, build duration, failed deployments, review comments, and operational monitoring. Those findings can then influence future ticket sizing, testing practices, pipeline design, and automation.

Ultimately, the goal should not be to create the most automated pipeline possible. It should be to create a pipeline the team trusts. SummitWorks should make work smaller at the source, automate predictable quality gates, make staging deployment repeatable, and introduce intelligent agents only where they demonstrably reduce friction while preserving human judgment.

References:

GitHub. (2026). **Building and testing Node.js**. GitHub Docs. https://docs.github.com/en/actions/tutorials/build-and-test-code/nodejs

GitHub. (2026). **Deployment environments**. GitHub Docs. https://docs.github.com/en/actions/concepts/workflows-and-actions/deployment-environments

GitHub. (2026). **Status checks**. GitHub Docs. https://docs.github.com/en/pull-requests/reference/status-checks

Krasso, Richard (2026). “WEB 430 Weekly Reading”. Weeks 1–5. Bellevue University

Part II: SummitWorks Development Pipeline Analysis

Steps 1 thru 5: Use Deepseek to analyze code snippet

Pull Request Summary Response from Deepseek

Core Changes: Two date utility functions added to handle date formatting and day difference calculations.

📝 Function Overview

`formatDate(dateString)`

- **Purpose**: Converts a date string to ISO format (YYYY-MM-DD)
- **Key Logic**:
  - Creates Date object from input string
  - **Applies timezone workaround**: Adds 1 hour (3600000ms) to compensate for timezone issues
  - Returns date portion only (removes time component)

`daysBetween(start, end)`

- **Purpose**: Calculates absolute number of days between two dates
- **Key Logic**:
  - Creates Date objects from both inputs
  - Calculates absolute difference in milliseconds
  - Converts to days (dividing by milliseconds per day)

⚠️ Potential Issues & Risks

Critical Issues

1. **Timezone "Fix" is Dangerous**
   - Hardcoded 1-hour offset assumes specific timezone issue
   - Will break for different timezone scenarios
   - No documentation explaining what issue this "fixes"
2. **Input Validation Missing**
   - Both functions accept any string → `new Date("invalid")` returns `Invalid Date`
   - No error handling for malformed dates
   - `daysBetween` with invalid dates returns `NaN`
3. **Date Calculation Precision**
   - `daysBetween` uses floating-point division → may return fractional days
   - Depends on exact time of day in input strings

Code Quality Concerns

- No input parameter validation (mentioned in TODO comment)
- Magic number `3600000` without explanation
- No unit tests included

- Timezone handling is a temporary "fix" without long-term solution

🛠️ Suggested Improvements

Immediate Fixes

1. **Remove Timezone Workaround**

```
export function formatDate(dateString) {
  const date = new Date(dateString);
  // Use UTC methods to avoid timezone issues
  return `${date.getUTCFullYear()}-${pad(date.getUTCMonth()+1)}-${pad(date.getUTCDate())}`;
}
```

6. **Add Input Validation**

```
function isValidDate(date) {
  return date instanceof Date && !isNaN(date);
}

export function daysBetween(start, end) {
  const s = new Date(start);
  const e = new Date(end);

  if (!isValidDate(s) || !isValidDate(e)) {
    throw new Error('Invalid date input');
  }

  return Math.floor(Math.abs(e - s) / (1000 * 60 * 60 * 24));
}
```

Enhanced Solutions

1. **Use Date Library**
   - Consider `date-fns` or `dayjs` for robust date handling
   - Eliminates manual timezone and edge case handling
2. **Add Configuration**
   - Make timezone offset configurable if needed
   - Add environment-specific settings
3. **Complete the TODO**
   - Implement validation as noted in comment
   - Add JSDoc documentation
   - Include unit tests for edge cases

🤖 Code Review Agent Support Points

Automated Detection

- **Static Analysis**: Flag magic numbers, missing validation
- **Security Scanning**: Identify potential injection risks
- **Pattern Recognition**: Detect known anti-patterns (hardcoded timezone fixes)

Workflow Integration

1. **Pre-commit Hooks**
   - Run automated checks before code submission
   - Validate date handling patterns
2. **Pull Request Analysis**

    - Generate automatic feedback on date-related code
    - Suggest alternative implementations
    - Flag TODOs and temporary fixes
3. **Knowledge Base Integration**
    - Link to company's date handling best practices
    - Provide examples of proper timezone management
    - Suggest relevant libraries based on project context

Context-Aware Suggestions

- **Project-Specific**: Check if date library already in dependencies
- **Team Standards**: Enforce consistent error handling patterns
- **Historical Context**: Reference similar fixes in codebase history

---

🎯 **Recommendation Priority**: Address the timezone workaround first, as it's the most risky implementation detail that could cause subtle bugs across different environments.

AI-Generated Pull Request Summary

DeepSeek identified two new date utility functions in the pull request.

The `formatDate(dateString)` function converts a supplied date string into an ISO-style `YYYY-MM-DD` value. It creates a JavaScript `Date` object and then adds one hour as a temporary workaround for a timezone issue before returning only the date portion of the ISO string.

`The daysBetween(start, end)` function creates `Date` objects from two supplied values, calculates the absolute difference between them in milliseconds, and converts that difference into days.

AI-Generated Issues and Risks

DeepSeek identified several potential concerns with the implementation:

- The hardcoded one-hour timezone adjustment is fragile because it assumes that the same offset will correctly solve every timezone-related situation.
- Neither function validates its input. Invalid date strings could result in an `Invalid Date` object or a `NaN` result.
- The `daysBetween()` calculation can return fractional days depending on the time values supplied.
- The value `3600000` is a magic number and does not clearly communicate its purpose.
- The code contains a TODO for validation but does not implement it.
- No unit tests are shown for expected behavior or edge cases.
- The temporary timezone workaround lacks enough documentation to explain the original problem it is intended to solve.

AI-Generated Suggested Improvements

DeepSeek recommended removing the hardcoded timezone workaround and using a more deliberate UTC-based approach to date formatting. It also recommended adding validation to ensure that supplied values produce valid JavaScript `Date` objects.

For `daysBetween()`, DeepSeek suggested validating both dates before performing the calculation and explicitly deciding whether the function should return whole or fractional days. It also recommended replacing unexplained magic values with clearer logic or named constants.

Additional recommendations included adding JSDoc documentation, implementing the existing validation TODO, creating unit tests for edge cases, and considering an established date library such as `date-fns` or `dayjs` if the application requires more complex date and timezone handling.

Where a Code Review Agent Could Help

DeepSeek suggested that a code review agent could provide an automated first review of changes such as this one. The agent could flag missing validation, TODO comments, magic numbers, and potentially risky date or timezone handling before a human reviewer begins examining the pull request.

An agent could also compare the change with project-specific standards, determine whether the project already uses a date-handling library, and identify similar changes elsewhere in the repository. It could generate initial review comments and draw attention to files or logic that deserve closer inspection.

This type of agent would be most useful as a support mechanism rather than as the final authority on whether the change should be accepted.

Reflection

I thought DeepSeek did a good job of quickly identifying the main purpose of both functions and explaining the pull request in language that would be useful to a reviewer. Its strongest observation was the hardcoded one-hour timezone adjustment because that kind of temporary fix could easily create subtle problems in different environments or timezones. It also correctly noticed that input validation was missing even though the code itself contains a TODO acknowledging that problem. I found the comments about invalid dates, fractional day calculations, and missing unit tests practical because they point directly toward areas that should be clarified before the code is merged.

The suggestion that a code review agent could flag TODOs, magic numbers, and missing validation also seems realistic and could save a reviewer time during the initial pass. However, I would not automatically accept DeepSeek's recommendation to replace the timezone workaround with UTC logic because the AI does not know the actual business requirement for how dates are supposed to be interpreted. Its recommendation to use `Math.floor()` for the day calculation also assumes that the application wants whole days, which may or may not be correct. I also thought the suggestion about security scanning and injection risk was less relevant to this particular code than the validation and timezone issues it identified. Recommending an external date library could be reasonable, but adding another dependency should be justified by the complexity of the application's actual date-handling requirements.

Overall, the recommendations would be useful to a real development team because they give the reviewer a strong starting point and highlight areas that deserve deeper investigation. This exercise reinforces the idea from the Week 5 readings that an intelligent code review agent can reduce cognitive load and improve consistency, but it cannot replace the business context, architectural knowledge, and judgment of the human reviewer. In the Final Project, I would want an AI-augmented workflow to use agents in this same supporting role: automatically surface likely problems and relevant context, while leaving consequential decisions with the development team.

Reference:

Krasso, Richard (2026). ”WEB 430 Weekly Reading”. Week 5. Bellevue University.